

A High-Throughput, Field Programmable Gate Array Implementation of Soft Output Viterbi Algorithm for Magnetic Recording

Lingyan Sun, Toshihiro Horigome, and B. V. K. Vijaya Kumar, *Senior Member*

Abstract—A high throughput, reconfigurable field programmable gate array (FPGA) implementation of the soft output Viterbi algorithm (SOVA) is described. Such a SOVA module provides the soft information needed by an iterative soft decoder used with Turbo codes and low density parity check (LDPC) codes. The implementation of the SOVA algorithm runs at 100 Mb/s on Xilinx Virtex II 2000 FPGA. Using a higher capacity FPGA and repeating the circuit doubles the throughput. The design can be reconfigured for different partial response (PR) targets. The design requires about 5 kb of memory for the EPR4 channel and may be integrated on a single chip with serially implemented LDPC decoder.

Index Terms—Decoder, LDPC codes, SOVA.

I. INTRODUCTION

WITH THE rediscovery of low-density parity check (LDPC) codes [1], there has been intense interest in how to apply the LDPC codes to the magnetic recording channel. In order to obtain the potential signal-to-noise ratio (SNR) benefits offered by the LDPC codes for magnetic recording partial response (PR) channels, the Viterbi algorithm (VA) used in the conventional partial response maximum likelihood (PRML) channels must be replaced by a channel detector providing soft information (e.g., log likelihood ratio (LLR) which is the logarithm of $\text{Prob}(\text{bit} = 1)/\text{Prob}(\text{bit} = 0)$). Soft output Viterbi algorithm (SOVA) [2] is a good candidate for the soft information channel detector. Its complexity is lower than that of the maximum a posteriori probability (MAP) detector [3], and the SNR loss is less than 0.3 dB [2].

Significant effort has been devoted to implementing SOVA in the form of an application specific integrated circuit (ASIC) [4]–[6]. Though the ASIC design of SOVA may provide better performance in speed and area, for fast and flexible evaluation of different LDPC codes for partial response (PR) channels (of interest in data storage systems), a field programmable gate array (FPGA) implementation provides the needed speed and flexibility. For FPGA implementation, limited resources of FPGA

give fewer degrees of freedom compared to ASIC. Also for future migration to higher capacity FPGA, a scalable design is preferred to achieve higher throughput in FPGA with higher capacity.

We designed a SOVA decoder for FPGA to be concatenated to an LDPC decoder for channel evaluation. This SOVA decoder can be integrated with the LDPC decoder to investigate how LDPC codes can be used for magnetic recording applications.

The rest of this paper is organized as follows. After introducing three different SOVA (classic, two-step and bi-directional) architectures in Section II, we present the circuit architecture and design details in Section III. Section IV is devoted to testing methodologies, results, and performance evaluation. Finally, we conclude the paper in Section V.

II. SOVA ARCHITECTURES

SOVA was proposed in 1989 by Hagenauer [2]. It is a modification of the VA to produce the reliability information for every bit decision. As in VA, all the calculations in SOVA are based on a trellis with the number of states depending on the length of the PR target. However, in SOVA, every state is assigned soft information, which is updated by the probability of selection error every time a decision is made along the path and when the information bits of two candidate paths are different. The algorithm outputs the updated soft information to indicate the confidence in bit decision. The first version of SOVA (also called classic SOVA) suffers from high complexity due to the need to update LLR for every state, as the maximum-likelihood (ML) path is unknown when performing the calculation.

Two-step SOVA is a method to reduce the complexity of the calculation associated with the classic SOVA [7]. The idea is that since we only output the LLR along the ML path, the traceback can be done along the ML path states instead of all the states. As shown in Fig. 1, this approach looks forward an extra distance d using the normal VA algorithm in order to find the ML path and trace back along the ML path to find the LLR for every bit decision. The complexity can be greatly reduced in this case. The main bottleneck is in implementing the traceback. For outputting one symbol, we need to trace back many states. Thus, to keep high throughput, each state must have access to all the information about the path metric differences and decision vectors. One solution is using orthogonal memory access as in [5]. The memory size and throughput is main concern with the large number of ports. Another solution is using systolic array processing, with path matrices and decision vectors propagating

Manuscript received December 15, 2003.

L. Sun and B. V. K. Vijaya Kumar are with the Data Storage Systems Center, Department of Electrical and Computer Engineering, Carnegie Mellon University, Pittsburgh, PA 15213 USA (e-mail: lsun@ece.cmu.edu; kumar@ece.cmu.edu).

T. Horigome is with the Data Storage Systems Center, Department of Electrical and Computer Engineering, Carnegie Mellon University, Pittsburgh, PA 15213 USA, and also with Sony Corporation, Tokyo 141-0001, Japan (e-mail: Toshihiro.Horigome@jp.sony.com).

Digital Object Identifier 10.1109/TMAG.2004.832672

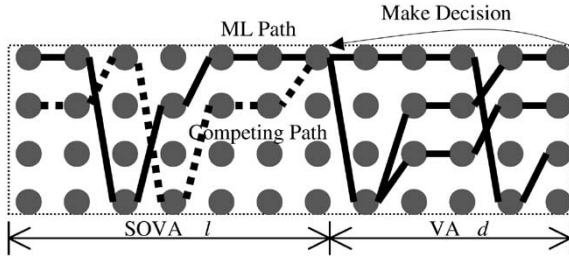


Fig. 1. Two-step SOVA.

along register pipeline. The register count is quite large and so is the power consumption, as a large amount of data needs to be shifted between registers in every cycle.

The traceback bottleneck imposes a more serious problem for FPGA implementation than for an ASIC. In FPGA implementation, transistor level design to form the orthogonal memory as in [5] is not possible and the number of registers is also quite limited. Block memory in FPGA can be used to store all the intermediate results. A major benefit of FPGA is that it is re-configurable. A module that can be repeatedly instantiated for parallel processing to meet certain throughput requirement is preferable. Bidirectional SOVA is considered as a better choice in this case.

In bidirectional SOVA [8], calculation is done in both forward and backward directions as shown in Fig. 2. It first goes forward to find ML path and outputs forward path metric (PM). Then it starts from the end of the block to go backward repeating the same calculation as in the forward processing to output backward PM and branch metric (BM). The LLR of the whole block can be calculated with the knowledge of the forward PM, backward PM and BM using

$$\begin{aligned} \text{LLR}(u_k) &= P_{\text{ML}_k} - P_{\text{competitor}_k} \\ P_{\text{ML}_k} &= \text{PM}_{\text{ML}_{f_k}} + \text{PM}_{\text{ML}_{b_k}} \\ P_{\text{competitor}_k} &= \min_{u_i \neq u_k} (\text{PM}_{f_{i-1}} + \text{BM}_{i-1} + \text{PM}_{b_i}). \end{aligned} \quad (1)$$

The LLR of the k th user bit is the PM difference between ML path and its most competitive path. The PM of ML path is the summation of its forward PM and backward PM. The most competitive path is the path with the smallest PM within all the paths with different bit decision from the ML path.

In this scheme, data are divided into blocks, and parallel processing between blocks can be used provided sufficient hardware resources exist. Also, it is not required to trace back many states for a single symbol. The throughput can be increased without running into the problem of the two-step SOVA as mentioned before.

III. FPGA IMPLEMENTATION

We use the bidirectional architecture for FPGA implementation. The core of the SOVA circuit (see Fig. 3) includes a calculation unit and a memory unit. The calculation unit has one forward calculation unit (FCU), two backward calculation units (BCU), and a LLR calculation unit (LLRU).

In the calculation unit, the FCU goes forward along the trellis, calculating and recording the forward PM and trace in memory.

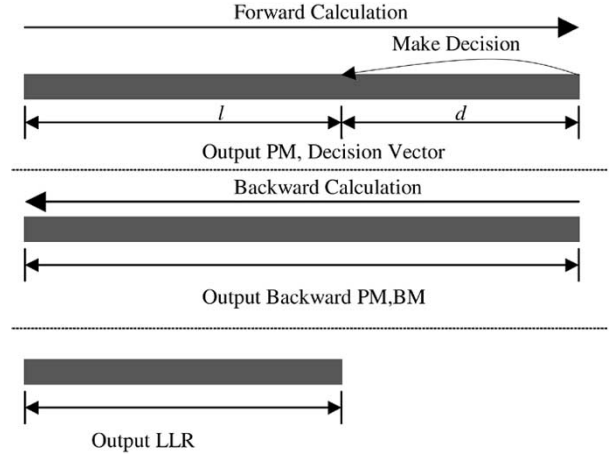


Fig. 2. Bidirectional SOVA.

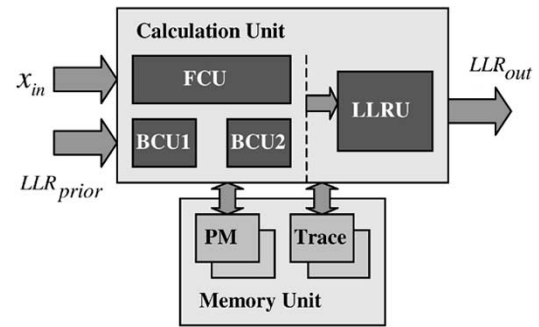


Fig. 3. Circuit architecture and modules.

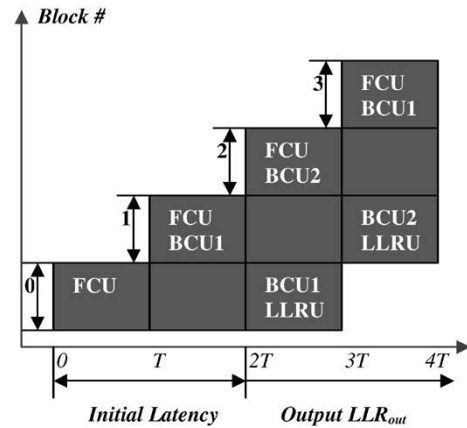


Fig. 4. Circuit scheduling.

After distance l as shown in Fig. 2, it starts to record the initial states for the survivor paths. When FCU finishes the extra length d , it finds out the ML path within one clock cycle. In the circuit, d and l are chosen to be the same. The received data are divided into blocks with one block of length l . The two BCU go backward along the trellis and do calculation for two consecutive blocks with one BCU calculating the LLR for the first block and the other looking forward for the ML path in the VA region for the next block. Fig. 4 shows the parallel operation of FCU, BCU1, BCU2, and LLRU. The x direction is for time and the y direction is the block number. This scheduling is to cancel the overhead of the extra distance for finding ML path and thereby increase the throughput.

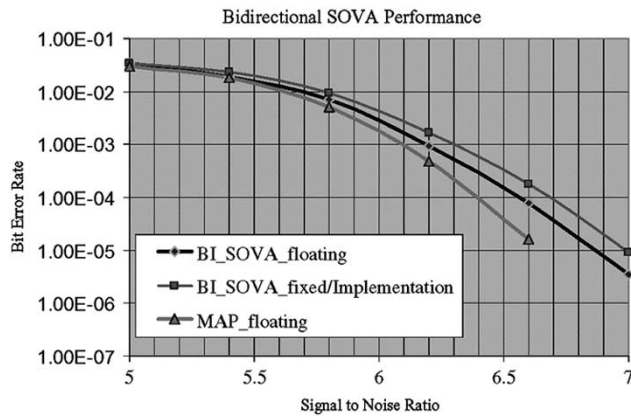


Fig. 5. SOVA + LDPC decoder performance.

The LLRU gets forward PM, trace from memory, and backward PM and BM from the backward calculation units. It is fully pipelined for tracing back and calculating the LLR according to (1). The memory is shared between FCU, BCU1, BCU2, LLRU and data blocks along the time. For EPR4 channel, 250 bits memory to store read back signal and 4 kb memory to store PMs and traces are required with trellis depth l set to 15. To simplify reconfiguration for other PR targets, extra memory size is used in the circuit.

Eight-bit representations are used for the received signal, *a priori* information and the final LLR, whereas 16 bits are used for intermediate results. The fixed-point representation gives similar bit-error rate (BER) results as the floating-point one. Fig. 5 shows the performance of the implemented fixed-point SOVA with LDPC decoder compared to the floating-point SOVA and MAP channel detector with LDPC decoder. At BER 10^{-5} , the performance of the implemented fixed point SOVA + LDPC is about 0.4 dB worse than that of MAP + LDPC, within which around 0.1 dB loss is due to the fixed-point implementation.

IV. RESULTS

SOVA decoder is written in VHDL and tested on the Xilinx Virtex II 2000 FPGA. The testing platform is shown in Fig. 6. A gold C model using same integer representation serves as the reference during debugging and testing for values of PM, BM, decision vector and final LLR.

The final circuit can run at 100 Mb/s for EPR4 configuration.

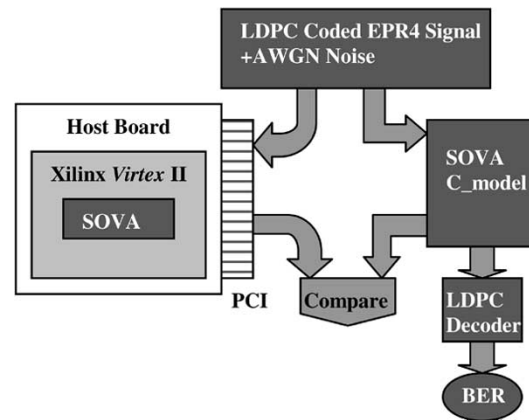


Fig. 6. Testing platform.

V. CONCLUSION

In this paper, we described the FPGA implementation of SOVA decoder, which is targeted to be connected with a serially implemented LDPC decoder for LDPC-encoded magnetic recording channel evaluation. The architecture was selected after evaluating three different schemes of SOVA decoding. By using a novel scheduling method, we can continuously output one bit per clock cycle after the initial delay. The circuit can be reconfigured for different PR targets, and can serve as a standard cell with the final throughput a linear function of the number of instantiated units.

REFERENCES

- [1] D. J. C. Mackay and R. M. Neal, "Good codes based on very sparse matrices," in *5th IMA Conf. Cryptography and Coding*, 1995.
- [2] J. Hagenauer and P. Hoeher, "A Viterbi algorithm with soft-decision outputs and its applications," in *Proc. Global Communications*, 1989, pp. 1680–1686.
- [3] L. Bahl, J. Cocke, F. Jelinek, and J. Raviv, "Optimal decoding of linear codes for minimizing symbol error rate," *IEEE Trans. Inform. Theory*, vol. 20, pp. 284–287, Mar. 1974.
- [4] O. J. Joeressen and H. Meyr, "A 40 Mb/s soft-output Viterbi decoder," *IEEE J. Solid-State Circuits*, vol. 30, pp. 812–818, July 1995.
- [5] D. Garret and M. Stan, "A 2.5 Mb/s, 25 Mw SOVA traceback chip for turbo decoding application," in *IEEE Int. Symp. Circuits and Systems*, May 2001, pp. 61–64.
- [6] E. Yeo, S. Augsburger, W. R. Davis, and B. Nikolic, "Implementation of high throughput soft output Viterbi decoder," in *IEEE Workshop on Signal Processing Systems*, Oct. 2002, pp. 146–151.
- [7] C. Berrou, P. Adde, E. Angui, and S. Faudeil, "A low complexity soft-output viterbi decoder architecture," in *IEEE Int. Conf. Communications*, vol. 2, May 1993, pp. 737–740.
- [8] B. Vucetic and J. Yuan, *Turbo Codes: Principles and Applications*. Norwell, MA: Kluwer, 2000, ISBN0 792 378 687, pp. 126–137.